Secretaría de Educación

DIRECCIÓN GENERAL DE UNIVERSIDADES TECNOLÓGICAS y POLITÉCNICAS

Universidad Tecnológica de Tehuacán

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

Nombre del alumno:

Alexis Montalvo Osorio

8° "A"

Docente:

José Miguel Carrera Pacheco

Materia:

Desarrollo Web Profesional

CUATRIMESTRE ENERO-ABRIL 2026

¿Qué es autenticación y en qué se diferencia de autorización?

La autenticación es el proceso mediante el cual un sistema verifica la identidad de un usuario antes de permitirle acceder a la aplicación. Generalmente se realiza mediante credenciales como correo electrónico y contraseña.

Por ejemplo, cuando un usuario intenta iniciar sesión en una aplicación, el sistema compara los datos introducidos con los registrados en la base de datos. Si coinciden, el usuario es autenticado.

Por otro lado, la autorización determina qué acciones puede realizar un usuario dentro del sistema una vez que ya ha sido autenticado.

En otras palabras:

- **Autenticación:** verifica quién es el usuario.
- **Autorización:** determina qué puede hacer ese usuario.

Por ejemplo, un usuario normal puede crear reportes en el sistema, mientras que un administrador puede moderarlos o eliminarlos.

2. ¿Qué son JWT, sesiones y OAuth?

JWT (JSON Web Token)

JWT es un tipo de token utilizado para representar información de autenticación de forma segura entre el cliente y el servidor. Este token contiene información codificada sobre el usuario y está firmado digitalmente para evitar manipulaciones.

Ventajas principales:

- Permite autenticar usuarios sin consultar constantemente la base de datos.
- Es muy utilizado en APIs modernas.
- Facilita el desarrollo de sistemas escalables.

Sesiones

Las sesiones son un mecanismo tradicional de autenticación donde el servidor mantiene información sobre los usuarios que han iniciado sesión.

Funcionamiento básico:

1. El usuario inicia sesión.
2. El servidor crea una sesión.
3. El navegador almacena un identificador de sesión en una cookie.
4. Cada vez que el usuario hace una petición, el servidor valida esa sesión.

OAuth

OAuth es un protocolo que permite a una aplicación acceder a recursos de otro servicio sin compartir directamente las credenciales del usuario.

Ejemplos comunes:

- Iniciar sesión con Google
- Iniciar sesión con GitHub
- Iniciar sesión con Facebook

Este sistema delega la autenticación a proveedores externos.

3. ¿Cuándo conviene usar tokens y cuándo sesiones?

Ambos métodos tienen ventajas dependiendo del tipo de sistema.

Uso de Tokens (JWT)

Los tokens son recomendables cuando:

- Se desarrollan APIs modernas.
- Se utilizan arquitecturas distribuidas o microservicios.
- Se requiere escalabilidad.

Ventajas:

- No dependen del estado del servidor.
- Permiten autenticación rápida.

Uso de Sesiones

Las sesiones son más adecuadas cuando:

- Se trabaja con aplicaciones web tradicionales.

- Se requiere mayor control desde el servidor.

Ventajas:

- Permiten invalidar sesiones fácilmente.
- Mayor control sobre la seguridad.

4. Riesgos de seguridad en un mal diseño de autenticación

Un sistema de autenticación mal diseñado puede generar múltiples vulnerabilidades.

Algunos riesgos comunes incluyen:

Robo de sesión

Si un atacante obtiene el token o cookie de sesión, puede suplantar la identidad del usuario.

Ataques de fuerza bruta

Intentos repetidos de adivinar contraseñas hasta encontrar la correcta.

Fugas de información

Si los tokens o datos sensibles quedan expuestos en el frontend.

Escalada de privilegios

Cuando un usuario obtiene permisos de administrador sin autorización.

Para prevenir estos riesgos se utilizan medidas como:

- tokens con expiración
- cookies httpOnly
- validación de roles
- control de acceso en backend

Estas prácticas forman parte de una arquitectura segura de autenticación.

Arquitectura de autenticación y…

5. ¿Cómo se definen roles y permisos en sistemas reales?

En aplicaciones reales, los roles permiten definir qué acciones puede realizar cada tipo de usuario.

En el sistema **UrbanPulse** se plantean dos roles principales.

Usuario (USER)

Puede:

- crear reportes
- comentar reportes
- votar reportes
- visualizar contenido público

Administrador (ADMIN)

Puede:

- moderar reportes
- eliminar contenido
- gestionar usuarios
- supervisar la plataforma

Este modelo permite mantener control sobre las acciones disponibles dentro del sistema.

6. Conclusión

Diseñar correctamente la arquitectura de autenticación y autorización es fundamental para garantizar la seguridad de una aplicación.

Antes de comenzar el desarrollo es importante definir:

- el mecanismo de autenticación,
- los roles de usuario,
- los permisos del sistema,
- y los flujos de acceso.

En el proyecto UrbanPulse se propone utilizar autenticación basada en **JWT junto con cookies seguras y control de roles**, lo cual permite crear una arquitectura segura, escalable y preparada para futuras implementaciones.